Workshop 2

Maria Lucía Castillo García

Índice de entregables

Evidencia.....2

.....2

Parte 1: Construir dos APIs 2

Parte 2: Docker compose 6

Parte 3: Reverse Proxy 7

Parte 4: Analizar la diferencia.....9

Parte 5: Transformar el ejercicio en Reverse Proxy + Load Balancer.....9

RETO 12

Diferencias entre ambos escenarios 20

Evidencia

```
casti@MLC MINGW64 ~/workshop1-docker
$ cd ~

casti@MLC MINGW64 ~
$ mkdir -p workshop2-docker/users workshop2-docker/products workshop2-docker/nginx

casti@MLC MINGW64 ~
$ cd workshop2-docker

casti@MLC MINGW64 ~/workshop2-docker
$ ls -R
.:
nginx/  products/  users/

./nginx:

./products:

./users:
```

Parte 1: Construir dos APIs

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch users/app.py

casti@MLC MINGW64 ~/workshop2-docker
$ notepad users/app.py
```

```python
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "users-local")

@app.get("/")
def home():
    return jsonify({
        "service": "users",
        "message": "Servicio de usuarios",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch users/requirements.txt

casti@MLC MINGW64 ~/workshop2-docker
$ notepad users/requirements.txt
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch users/Dockerfile

casti@MLC MINGW64 ~/workshop2-docker
$ notepad users/Dockerfile
```

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch products/app.py

casti@MLC MINGW64 ~/workshop2-docker
$ notepad products/app.py
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch products/Dockerfile

casti@MLC MINGW64 ~/workshop2-docker
$ notepad products/Dockerfile
```

```python
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "products-local")

@app.get("/")
def home():
    return jsonify({
        "service": "products",
        "message": "Servicio de productos",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch products/requirements.txt

casti@MLC MINGW64 ~/workshop2-docker
$ notepad products/requirements.txt
```

```
FROM python:3.12-slim

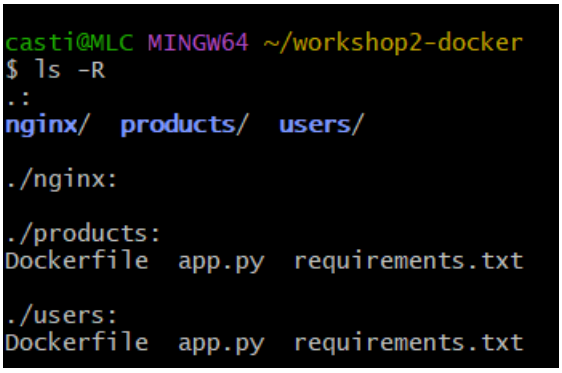
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```



```
casti@MLC MINGW64 ~/workshop2-docker
$ cat users/app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "users-local")

@app.get("/")
def home():
    return jsonify({
        "service": "users",
        "message": "Servicio de usuarios",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
casti@MLC MINGW64 ~/workshop2-docker
$ cat products/app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "products-local")

@app.get("/")
def home():
    return jsonify({
        "service": "products",
        "message": "Servicio de productos",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
casti@MLC MINGW64 ~/workshop2-docker
$
```

Parte 2: Docker compose

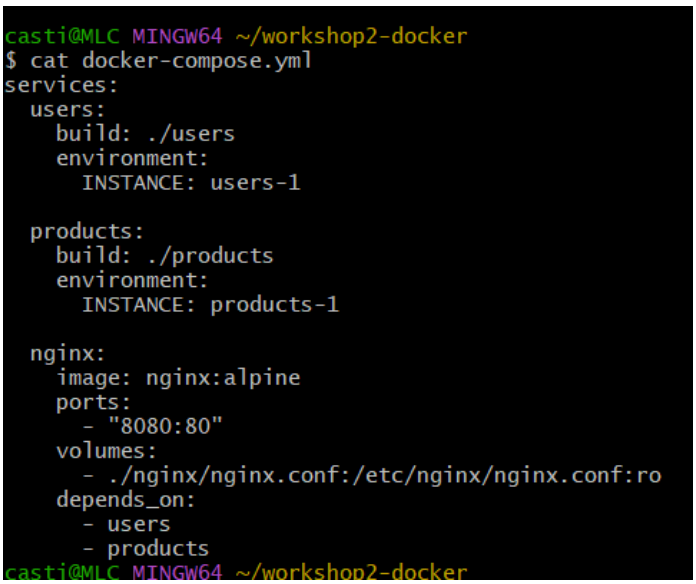
```
casti@MLC MINGW64 ~/workshop2-docker
$ touch docker-compose.yml

casti@MLC MINGW64 ~/workshop2-docker
$ notepad docker-compose.yml
```

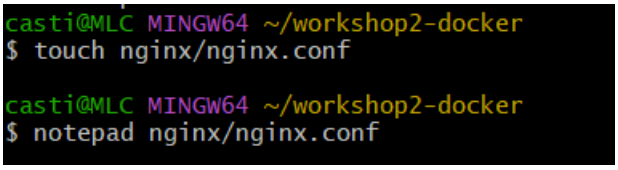
```yaml
services:
  users:
    build: ./users
    environment:
      INSTANCE: users-1

  products:
    build: ./products
    environment:
      INSTANCE: products-1

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - users
      - products
```



Parte 3: Reverse Proxy



```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    server {
        listen 80;

        location /api/users/ {
            proxy_pass http://users:5000/;
        }

        location /api/products/ {
            proxy_pass http://products:5000/;
        }
    }
}
```

Levantar y probar

```
casti@MLC MINGW64 ~/workshop2-docker
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.03kB 0.0s done
#1 DONE 0.0s

#2 [users internal] load build definition from Dockerfile
#2 transferring dockerfile: 215B 0.0s done
#2 DONE 0.1s

#3 [products internal] load build definition from Dockerfile
#3 transferring dockerfile: 215B done
#3 DONE 0.1s

#4 [products internal] load metadata for docker.io/library/python:3.12-slim
#4 DONE 1.2s

#5 [users internal] load .dockerignore
#5 transferring context: 2B done
#5 DONE 0.1s

#6 [products internal] load .dockerignore
#6 transferring context: 2B done
#6 DONE 0.1s

#7 [users 1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
#7 resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
#7 resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea 0.1s done
#7 DONE 0.1s

#8 [products internal] load build context
#8 transferring context: 442B done
#8 DONE 0.1s
```

...

```
#20 [products] resolving provenance for metadata file
#20 DONE 0.0s
[+] up 5/5
 ✔ Image workshop2-docker-users           Built
 ✔ Image workshop2-docker-products        Built
 ✔ Container workshop2-docker-products-1  Started
 ✔ Container workshop2-docker-users-1     Started
 ✔ Container workshop2-docker-nginx-1     Started
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ docker compose ps
NAME                          IMAGE                         COMMAND           SERVICE    CREATED          STATUS          PORTS
workshop2-docker-products-1   workshop2-docker-products   "python app.py"   products   25 seconds ago   Up 15 seconds   5000/tcp
workshop2-docker-users-1      workshop2-docker-users      "python app.py"   users      25 seconds ago   Up 15 seconds   5000/tcp
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ curl http://localhost:8080/api/users/
{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

casti@MLC MINGW64 ~/workshop2-docker
$ curl http://localhost:8080/api/products/
{"instance":"products-1","message":"Servicio de productos","service":"products"}

casti@MLC MINGW64 ~/workshop2-docker
$ curl http://localhost:5000/
curl: (7) Failed to connect to localhost port 5000 after 2224 ms: Could not connect to server
```

Parte 4: Analizar la diferencia

```
casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..6}; do curl -s http://localhost:8080/api/users/; echo; done
{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}
```

Parte 5: Transformar el ejercicio en Reverse Proxy + Load Balancer

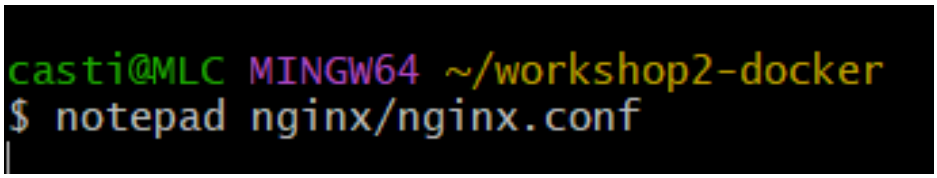
```
casti@MLC MINGW64 ~/workshop2-docker
$ notepad docker-compose.yml
```

```
services:
  users1:
    build: ./users
    environment:
      INSTANCE: users-1

  users2:
    build: ./users
    environment:
      INSTANCE: users-2

  products:
    build: ./products
    environment:
      INSTANCE: products-1

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - users1
      - users2
      - products
```



```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream users_backend {
        server users1:5000;
        server users2:5000;
    }

    server {
        listen 80;

        location /api/users/ {
            proxy_pass http://users_backend/;
        }

        location /api/products/ {
            proxy_pass http://products:5000/;
        }
    }
}
```

```
#22 [users2] resolving provenance for metadata file
#22 DONE 0.0s
[+] up 8/8
 ✓ Image workshop2-docker-users1         Built
 ✓ Image workshop2-docker-users2         Built
 ✓ Image workshop2-docker-products       Built
 ✓ Network workshop2-docker_default      Created
 ✓ Container workshop2-docker-products-1 Started
 ✓ Container workshop2-docker-users1-1   Started
 ✓ Container workshop2-docker-users2-1   Started
 ✓ Container workshop2-docker-nginx-1    Started

casti@MLC MINGW64 ~/workshop2-docker
$ docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE    CREATED         STATUS         PORTS
workshop2-docker-nginx-1    nginx:alpine                "/docker-entrypoint.…"   nginx      5 seconds ago   Up 4 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
workshop2-docker-products-1 workshop2-docker-products   "python app.py"          products   5 seconds ago   Up 4 seconds   5000/tcp
workshop2-docker-users1-1   workshop2-docker-users1     "python app.py"          users1     5 seconds ago   Up 4 seconds   5000/tcp
workshop2-docker-users2-1   workshop2-docker-users2     "python app.py"          users2     5 seconds ago   Up 4 seconds   5000/tcp
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ cat nginx/nginx.conf
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream users_backend {
        server users1:5000;
        server users2:5000;
    }

    server {
        listen 80;

        location /api/users/ {
            proxy_pass http://users_backend/;
        }

        location /api/products/ {
            proxy_pass http://products:5000/;
        }
    }
}
```

```
}
casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..6}; do curl -s http://localhost:8080/api/users/; echo; done
{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-2","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-2","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-2","message":"Servicio de usuarios","service":"users"}
```

RETO

PARTE A

```
casti@MLC MINGW64 ~/workshop2-docker
$ mkdir orders

casti@MLC MINGW64 ~/workshop2-docker
$ touch orders/app.py

casti@MLC MINGW64 ~/workshop2-docker
$ notepad orders/app.py
```

```python
import os
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "orders-local")

@app.get("/")
def home():
    return jsonify({
        "service": "orders",
        "message": "Servicio de ordenes",
        "instance": INSTANCE
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch orders/requirements.txt

casti@MLC MINGW64 ~/workshop2-docker
$ notepad orders/requirements.txt
```

```
Flask
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ touch orders/Dockerfile
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ notepad orders/Dockerfile
```

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ notepad docker-compose.yml
```

```
services:
  users1:
    build: ./users
    environment:
      INSTANCE: users-1

  users2:
    build: ./users
    environment:
      INSTANCE: users-2

  products:
    build: ./products
    environment:
      INSTANCE: products-1

  orders:
    build: ./orders
    environment:
      INSTANCE: orders-1

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - users1
      - users2
      - products
      - orders
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ notepad nginx/nginx.conf
```

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream users_backend {
        server users1:5000;
        server users2:5000;
    }

    server {
        listen 80;

        location /api/users/ {
            proxy_pass http://users_backend/;
        }

        location /api/products/ {
            proxy_pass http://products:5000/;
        }

        location /api/orders/ {
            proxy_pass http://orders:5000/;
        }

    }
}
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.97kB 0.0s done
#1 DONE 0.0s

#2 [products internal] load build definition from Dockerfile
#2 transferring dockerfile: 215B done
#2 DONE 0.0s

#3 [users2 internal] load build definition from Dockerfile
#3 transferring dockerfile: 215B 0.0s done
#3 DONE 0.0s

#4 [orders internal] load build definition from Dockerfile
#4 transferring dockerfile: 215B done
```

...

```
[+] up 9/9
 ✔ Image workshop2-docker-users1          Built
 ✔ Image workshop2-docker-users2          Built
 ✔ Image workshop2-docker-products        Built
 ✔ Image workshop2-docker-orders          Built
 ✔ Container workshop2-docker-nginx-1     Running
 ✔ Container workshop2-docker-orders-1    Started
 ✔ Container workshop2-docker-products-1  Started
 ✔ Container workshop2-docker-users1-1    Started
 ✔ Container workshop2-docker-users2-1    Started
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE    CREATED          STATUS          PORTS
workshop2-docker-nginx-1    nginx:alpine                "/docker-entrypoint.…"   nginx      23 minutes ago   Up 23 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
workshop2-docker-orders-1   workshop2-docker-orders     "python app.py"          orders     22 seconds ago   Up 11 seconds   5000/tcp
workshop2-docker-products-1 workshop2-docker-products   "python app.py"          products   22 seconds ago   Up 11 seconds   5000/tcp
workshop2-docker-users1-1   workshop2-docker-users1     "python app.py"          users1     22 seconds ago   Up 11 seconds   5000/tcp
workshop2-docker-users2-1   workshop2-docker-users2     "python app.py"          users2     22 seconds ago   Up 11 seconds   5000/tcp
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ curl http://localhost:8080/api/orders/
{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..6}; do curl -s http://localhost:8080/api/orders/; echo; done
{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}
```

PASO B

```
casti@MLC MINGW64 ~/workshop2-docker
$ notepad docker-compose.yml
```

```yaml
services:
  users1:
    build: ./users
    environment:
      INSTANCE: users-1

  users2:
    build: ./users
    environment:
      INSTANCE: users-2

  products:
    build: ./products
    environment:
      INSTANCE: products-1

  orders1:
    build: ./orders
    environment:
      INSTANCE: orders-1

  orders2:
    build: ./orders
    environment:
      INSTANCE: orders-2

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - users1
      - users2
      - products
      - orders1
      - orders2
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ notepad nginx/nginx.conf
```

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream users_backend {
        server users1:5000;
        server users2:5000;
    }

    upstream orders_backend {
        server orders1:5000;
        server orders2:5000;
    }

    server {
        listen 80;

        location /api/users/ {
            proxy_pass http://users_backend/;
        }

        location /api/products/ {
            proxy_pass http://products:5000/;
        }

        location /api/orders/ {
            proxy_pass http://orders_backend/;
        }

    }
}
```

```
#32 [users1] resolving provenance for metadata file
#32 DONE 0.0s
[+] up 12/12
 ✓ Image workshop2-docker-orders1        Built
 ✓ Image workshop2-docker-orders2        Built
 ✓ Image workshop2-docker-users1         Built
 ✓ Image workshop2-docker-users2         Built
 ✓ Image workshop2-docker-products       Built
 ✓ Network workshop2-docker_default      Created
 ✓ Container workshop2-docker-products-1 Started
 ✓ Container workshop2-docker-orders1-1  Started
 ✓ Container workshop2-docker-users1-1   Started
 ✓ Container workshop2-docker-users2-1   Started
 ✓ Container workshop2-docker-orders2-1  Started
 ✓ Container workshop2-docker-nginx-1    Started

casti@MLC MINGW64 ~/workshop2-docker
$ docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE    CREATED         STATUS         PORTS
workshop2-docker-nginx-1    nginx:alpine                "/docker-entrypoint.…"   nginx      9 seconds ago   Up 8 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
workshop2-docker-orders1-1  workshop2-docker-orders1    "python app.py"          orders1    9 seconds ago   Up 9 seconds   5000/tcp
workshop2-docker-orders2-1  workshop2-docker-orders2    "python app.py"          orders2    9 seconds ago   Up 9 seconds   5000/tcp
workshop2-docker-products-1 workshop2-docker-products   "python app.py"          products   9 seconds ago   Up 9 seconds   5000/tcp
workshop2-docker-users1-1   workshop2-docker-users1     "python app.py"          users1     9 seconds ago   Up 9 seconds   5000/tcp
workshop2-docker-users2-1   workshop2-docker-users2     "python app.py"          users2     9 seconds ago   Up 9 seconds   5000/tcp
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ cat nginx/nginx.conf
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream users_backend {
        server users1:5000;
        server users2:5000;
    }

    upstream orders_backend {
        server orders1:5000;
        server orders2:5000;
    }

    server {
        listen 80;

        location /api/users/ {
            proxy_pass http://users_backend/;
        }

        location /api/products/ {
            proxy_pass http://products:5000/;
        }

        location /api/orders/ {
            proxy_pass http://orders_backend/;
        }

    }
}
casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..6}; do curl -s http://localhost:8080/api/orders/; echo; done
{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-2","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-2","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-2","message":"Servicio de ordenes","service":"orders"}
```

```
casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..4}; do curl -s http://localhost:8080/api/users/; echo; done
{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-2","message":"Servicio de usuarios","service":"users"}

{"instance":"users-1","message":"Servicio de usuarios","service":"users"}

{"instance":"users-2","message":"Servicio de usuarios","service":"users"}


casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..4}; do curl -s http://localhost:8080/api/products/; echo; done
{"instance":"products-1","message":"Servicio de productos","service":"products"}

{"instance":"products-1","message":"Servicio de productos","service":"products"}

{"instance":"products-1","message":"Servicio de productos","service":"products"}

{"instance":"products-1","message":"Servicio de productos","service":"products"}


casti@MLC MINGW64 ~/workshop2-docker
$ for i in {1..4}; do curl -s http://localhost:8080/api/orders/; echo; done
{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-2","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-1","message":"Servicio de ordenes","service":"orders"}

{"instance":"orders-2","message":"Servicio de ordenes","service":"orders"}
```

Diferencias entre ambos escenarios

Criterio	Reverse proxy + routing (Parte 3)	Reverse proxy + load balancing (Parte 5)
¿Qué decide Nginx?	A qué servicio enviar la petición, según la ruta.	A qué *instancia* del servicio enviarla, entre varias iguales.
Backends por ruta	Uno solo	Varios intercambiables
Configuración	`proxy_pass` apunta directo al servicio: `http://users:5000/`	`proxy_pass` apunta a un grupo: `http://users_backend/`, definido en un bloque upstream
Criterio de selección	La ruta solicitada (`/api/users/` → `users`)	Round Robin entre los miembros del upstream

Resultado observado	Seis peticiones, seis veces `users-1`	Seis peticiones alternando `users-1` y `users-2`
Si el backend cae	La ruta deja de funcionar porque no hay a quién más enviar	El servicio sigue, porque Nginx reenvía a la instancia que sí responde
Problema que resuelve	Punto de entrada único y ocultamiento de los puertos internos	Distribución de carga y tolerancia a fallos
Tipo de escalamiento	Ninguno porque organiza, pero no amplía la capacidad	Horizontal: la capacidad crece agregando instancias

La diferencia clave es que no son mecanismos alternativos, sino uno construido sobre el otro. El reverse proxy es la base: un único punto de entrada que recibe todas las peticiones y decide a qué servicio interno enviarlas, ocultando los puertos y la cantidad de servicios que existen detrás. El balanceo es una capa adicional que solo aparece cuando una misma ruta tiene varias instancias disponibles y hay que elegir entre ellas.

Por eso en la Parte 3 hay reverse proxy pero no balanceo: cada ruta tenía un único destino posible, así que no había nada que decidir. En la Parte 5 el nginx.conf quedó con los dos comportamientos conviviendo (/api/users/ balanceada entre dos instancias y /api/products/ apuntando a una sola), lo que muestra que un reverse proxy puede o no balancear según cómo esté configurada cada ruta.